

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 05 -

ALTA DISPONIBILIDADE (HA) E SEGURANÇA

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	22
O QUE VOCÊ VIU NESTA AULA?	23
REFERÊNCIAS.....	24

EMANIP

O QUE VEM POR AÍ?

No dinâmico universo da tecnologia, a falha de um sistema pode custar milhões e abalar a confiança de usuários. Alta Disponibilidade e Segurança não são mais luxos, mas pilares inegociáveis para qualquer arquitetura web robusta. Você já se perguntou o que acontece quando o inesperado ocorre? Como garantir que sua aplicação permaneça on-line, performática e protegida contra as ameaças cibernéticas mais astutas?

Neste material, desvendaremos essas questões. Desde a eliminação de pontos únicos de falha e a automação de failovers inteligentes, até a blindagem de aplicações com Web Application Firewalls (WAFs) e a adoção de boas práticas de segurança, você aprenderá a projetar, implementar e gerenciar sistemas que prosperam em um ambiente digital desafiador.

HANDS ON

Nesta aula, você será conduzido(a) por um experimento prático que integra segurança à alta disponibilidade. O primeiro passo será preparar um ambiente com Nginx + ModSecurity, utilizando uma imagem customizada em Docker. A partir daí, você habilitará o OWASP Core Rule Set (CRS) e compreenderá como as regras padrão já oferecem uma camada sólida de proteção contra ataques comuns, como SQL Injection e Cross-Site Scripting.

Na sequência, criaremos regras personalizadas no ModSecurity para bloquear padrões específicos, como um User-Agent malicioso ou tentativas simples de injeção SQL. Essa prática vai mostrar como um Web Application Firewall pode ser ajustado para atender às necessidades do seu ambiente. Também exploraremos o recurso de rate limiting no Nginx, aplicando restrições em endpoints críticos, como /login, para mitigar ataques de força bruta e acessos excessivos.

Por fim, validaremos a segurança com testes reais de ataque e análise de métricas. Vamos simular requisições bloqueadas, acompanhar os códigos de status retornados (403 Forbidden) e consultar os logs de auditoria do ModSecurity para identificar quais regras foram acionadas. Essa etapa permitirá não apenas verificar a eficácia das defesas, mas também entender como ajustar os limiares de bloqueio e monitorar a operação em produção. O objetivo é que você saia desta prática dominando os fundamentos de como proteger aplicações web em um ambiente de alta disponibilidade.

```
curl http://localhost/login?user=admin' OR '1'='1
```

Código-fonte 1 - BASH Exemplo de simulação de ataque SQL Injection bloqueado pelo WAF
Fonte: Elaborado pelo autor (2025)

O código-fonte e os exemplos práticos de cada exercício estão organizados no [repositório](#) do curso:

SAIBA MAIS

A tecnologia permeia cada aspecto da nossa vida e do mundo dos negócios. Desde uma simples compra on-line até as transações financeiras mais complexas, a dependência de sistemas digitais que funcionam ininterruptamente é total. Mas, o que acontece quando esses sistemas falham? Como garantimos que nossa aplicação estará sempre acessível, performática e, crucialmente, protegida contra as ameaças que evoluem a cada segundo? É exatamente essa a jornada que vamos aprofundar agora, explorando os pilares da Alta Disponibilidade (HA) e da Segurança, elementos indissociáveis para qualquer arquitetura web moderna e robusta.

Desmistificaremos conceitos que vão muito além do jargão técnico, entendendo o impacto real das falhas, as promessas que fazemos aos nossos usuários por meio dos acordos de nível de serviço (SLA) e as metas internas que nos guiam (SLO). Abordaremos como identificar e neutralizar os famigerados Pontos Únicos de Falha (SPOFs), que são verdadeiros "calcanhares de Aquiles" em qualquer sistema. Nos aprofundaremos nas estratégias de HA, desde o tradicional modelo Ativo-Passivo até o dinâmico Ativo-Ativo, e como ferramentas de replicação e healthchecks avançados garantem que, mesmo quando o inesperado acontece, seu sistema se recupera de forma autônoma e inteligente.

Por fim, elevaremos o nível da proteção, mergulhando no mundo da segurança da aplicação. Conheceremos os Web Application Firewalls (WAFs), a primeira linha de defesa contra ataques direcionados à lógica da sua aplicação, e como motores poderosos como o ModSecurity, munidos do vasto conhecimento do OWASP Core Rule Set, podem blindar sua arquitetura. Finalizaremos com um arsenal de boas práticas de segurança, desde a criptografia robusta com TLS até o controle de tráfego com Rate Limiting, passando pela importância vital do monitoramento contínuo e dos testes de Recuperação de Desastres (DR). Preparem-se para solidificar seu conhecimento e transformar suas aplicações em verdadeiros baluartes de resiliência e segurança.

Alta Disponibilidade (HA): A Base da Continuidade do Negócio

A Alta Disponibilidade (HA) é a capacidade de um sistema de operar continuamente sem falhas por um determinado período. Em um mundo onde a interrupção de serviços digitais pode ter consequências catastróficas, o HA se tornou um requisito fundamental, não um diferencial. Pense em um e-commerce em plena Black Friday, um sistema de pagamentos global ou uma plataforma de comunicação vital em momentos de crise. Qualquer minuto fora do ar pode significar perdas financeiras astronômicas, danos irreparáveis à reputação e, em alguns casos, até impactos sociais.

O Cenário Atual: Por Que HA é Crítico?

Vivemos na era da disponibilidade "sempre ligada". Nossos usuários esperam que os serviços digitais estejam acessíveis a qualquer momento, de qualquer lugar. Quando isso não acontece, o resultado pode ser devastador:

Custo Financeiro:

- **Perda Direta de Receita:** transações interrompidas significam dinheiro não faturado. Um e-commerce pode perder milhares, ou milhões, de dólares por minuto de inatividade;
- **Perda de Produtividade:** colaboradores, clientes e parceiros ficam impossibilitados de realizar suas tarefas, gerando horas de trabalho perdidas;
- **Multas Contratuais:** acordos de Nível de Serviço (SLA) muitas vezes preveem penalidades financeiras pesadas para o provedor em caso de descumprimento dos níveis de disponibilidade prometidos;
- **Custos de Recuperação:** equipes de TI em plantão, horas extras, recuperação de dados e infraestrutura danificada geram despesas adicionais.

Impacto Reputacional:

- **Perda de Confiança:** clientes migram para a concorrência se o serviço não for confiável. A imagem de uma empresa "que sempre cai" é difícil de reverter;

- **Dano à Marca:** a reputação construída ao longo de anos pode ser comprometida em poucas horas de inatividade. Notícias negativas se espalham rapidamente pelas mídias sociais;
- **Desgaste Interno:** a equipe de tecnologia se vê sob pressão intensa durante incidentes, levando a estresse, desmotivação e, em casos extremos, à exaustão e rotatividade de talentos.

A meta da HA, portanto, é minimizar o **downtime** (tempo de inatividade) e garantir a **continuidade do negócio**, mesmo quando componentes individuais falham. É importante não confundir HA com **FaultTolerance**. Enquanto a FaultTolerance visa eliminar *completamente* o tempo de inatividade por meio de componentes duplicados e sincronizados (o que é extremamente caro), a HA foca em *minimizar* o impacto das falhas, buscando um equilíbrio entre custo e benefício.

SLA e SLO: A Linguagem da Confiabilidade

Para gerenciar as expectativas e medir a eficácia das estratégias de HA, utilizamos dois conceitos-chave:

SLA (Service Level Agreement - Acordo de Nível de Serviço): é um acordo formal e muitas vezes legalmente vinculante entre um provedor de serviço e um cliente. Ele define o nível de serviço esperado e, em caso de não cumprimento, pode acarretar em penalidades financeiras para o provedor.

- **Métricas comuns:** percentual de tempo que o serviço estará operacional (disponibilidade), tempo de resposta, taxa de erros, tempo para resolução de incidentes;

Exemplo: um provedor de nuvem pode garantir 99.99% de disponibilidade para uma máquina virtual. Isso equivale a aproximadamente 52 minutos de downtime por ano.

SLO (Service Level Objective - Objetivo de Nível de Serviço): é uma meta interna e mensurável que uma equipe define para si mesma, visando atingir o SLA. Não há penalidades financeiras diretas associadas ao seu não cumprimento, mas servem como indicadores de risco ao SLA.

- **Mais detalhado e granular:** enquanto o SLA é uma promessa de alto nível para o cliente, o SLO pode ter objetivos para subsistemas específicos que compõem o serviço geral;
- **Exemplo:** para garantir um SLA de 99.99% de disponibilidade do e-commerce, a equipe de banco de dados pode ter um SLO ainda mais ambicioso, como 99.999% de disponibilidade, pois a falha do banco de dados impactaria diretamente o e-commerce.

Característica	SLA (Service Level Agreement)	SLO (Service Level Objective)
Natureza	Acordo formal, promessa ao cliente.	Meta interna da equipe.
Vinculação	Legalmente vinculante, com penalidades.	Não vinculante, indicador de risco.
Público	Cliente.	Equipe de engenharia.
Detalhe	Alto nível, focado no serviço final.	Mais granular, focado em componentes.
Propósito	Gerenciar expectativas do cliente.	Guiar decisões técnicas e monitoramento.

Tabela 1 – Comparativo entre SLA e SLO
Fonte: elaborado pelo autor (2025)

Impacto no Negócio: a disciplina de Engenharia de Confiabilidade de Sites (SRE - Site Reliability Engineering) é construída em torno desses conceitos. SLA e SLO direcionam as decisões de engenharia, os investimentos em HA e a alocação de recursos, garantindo que o sistema seja confiável e que o negócio possa crescer sem medo de falhas.

Pontos Únicos de Falha (SPOF): Identificando Vulnerabilidades

Um **Ponto Único de Falha (SPOF)** é qualquer componente de um sistema que, se falhar, causa a interrupção completa do sistema. É o "calcanhar de Aquiles" de sua arquitetura, caracterizado pela ausência de redundância.

Exemplos Comuns de SPOFs:

Hardware: um único servidor web, um único banco de dados, um único switch de rede, uma única fonte de energia.

Software: um único serviço rodando em uma máquina, um único balanceador de carga.

Rede: uma única conexão de internet para o datacenter.

Localização: dependência de um único datacenter (em caso de desastre regional).

O objetivo principal das estratégias de HA é justamente eliminar ou mitigar o máximo de SPOFs possível por meio de redundância e mecanismos de failover.

SPOF em Balanceadores de Carga: A Proteção da Proteção

Balanceadores de carga (como Nginx, HAProxy, AWS ELB) são vitais para distribuir o tráfego e fornecer HA para os **back-ends** (servidores de aplicação). No entanto, um dilema comum surge: se o próprio balanceador de carga for um SPOF, toda a arquitetura de HA que ele proporciona pode ser comprometida. Por isso, é crucial garantir que o balanceador de carga seja ele próprio altamente disponível.

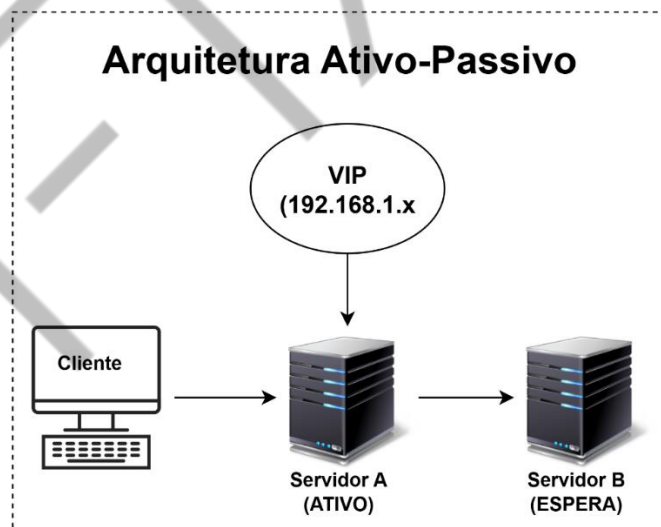


Figura 1 - Arquitetura Ativo-Pasivo
Fonte: elaborado pelo autor (2025)

Estratégias de HA: Ativo-Passivo vs. Ativo-Ativo

Para construir arquiteturas resilientes, existem duas estratégias principais de HA:

HA Ativo-Passivo (Active-Passive)

- **Conceito:** dois ou mais nós (servidores) para o mesmo serviço. Um nó está **ativo** (processando requisições) e o(s) outro(s) está(ão) **passivo(s)** (em espera, sincronizado, mas não processando requisições);
- **Funcionamento:** se o nó ativo falha, o nó passivo assume o papel de ativo;
- **Vantagens:** mais simples de configurar e gerenciar, evita problemas de concorrência de dados (pois apenas um está ativo por vez);
- **Desvantagens:** recursos ociosos (o nó passivo não está sendo utilizado para tráfego produtivo), podendo haver um pequeno tempo de interrupção durante a transição (failover);
- **Casos de Uso:** bancos de dados mestres, firewalls, e balanceadores de carga com IP virtual (como veremos com Keepalived).

HA Ativo-Ativo (Active-Active)

- **Conceito:** dois ou mais nós para o mesmo serviço, todos **ativos** e processando requisições simultaneamente;
- **Funcionamento:** um balanceador de carga distribui o tráfego entre todos os nós ativos. Se um nó falha, o tráfego é redistribuído entre os nós restantes;
- **Vantagens:** utilização máxima dos recursos (todos os nós trabalham), maior escalabilidade (aumenta a capacidade total do sistema), e menor tempo de failover (a interrupção é quase imperceptível);
- **Desvantagens:** mais complexo de configurar, gerenciar e sincronizar (especialmente para aplicações que mantêm estado), e pode haver desafios com conflito de dados ou consistência;
- **Casos de Uso:** servidores web sem estado (stateless), APIs, microsserviços.

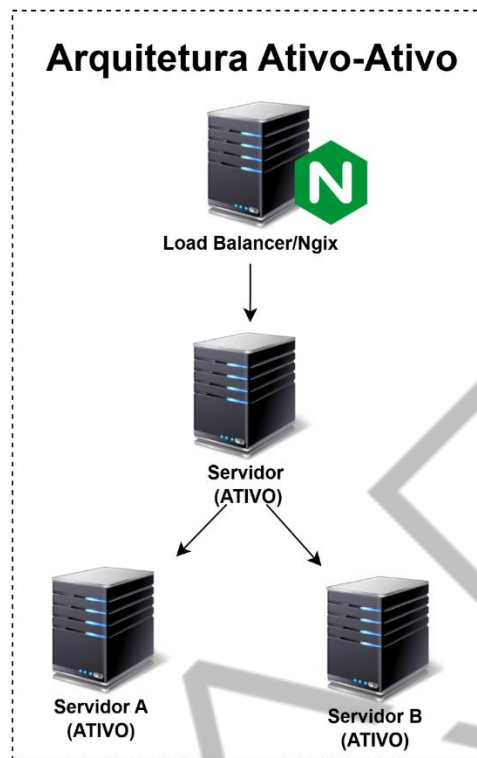


Figura 2 - Arquitetura Ativo-Ativo
Fonte: elaborado pelo autor (2025)

Construindo um Cluster HA Robusto: Além do Básico

Ter múltiplas instâncias de um serviço é apenas o primeiro passo para a HA. Para que a redundância seja efetiva, precisamos garantir que as configurações estejam sincronizadas e que o sistema seja capaz de detectar e reagir a falhas de forma autônoma e inteligente.

Replicação de Configuração: Mantendo a Consistência

Em um ambiente de HA, com múltiplos servidores atuando no mesmo papel (seja ativo-ativo ou ativo-passivo), é vital que todos eles possuam as **mesmas configurações**, arquivos de conteúdo, scripts, regras, etc. A inconsistência de configuração é uma das principais causas de falhas em failovers, onde o nó que assume o controle não se comporta como esperado, ou não possui todos os recursos necessários.

Por que é crucial?

Garantir o Failover Suave: se o nó de backup não tem a configuração idêntica ao primário, o serviço pode não funcionar corretamente após a transição.

Consistência do Serviço: evitar que diferentes usuários recebam respostas distintas de nós diferentes.

Simplicidade na Gestão: atualizar a configuração em um único lugar e replicar de forma automatizada.

Estratégias e Ferramentas para Replicação

rsync (Sincronização Ponto a Ponto): um utilitário de linha de comando robusto e eficiente para sincronizar arquivos e diretórios. Sua grande vantagem é ser **diferencial**, ou seja, ele transfere apenas as partes dos arquivos que foram alteradas, tornando o processo muito rápido.

- Pode ser agendado via cron ou acionado por eventos para manter diretórios como /etc/nginx ou /usr/share/nginx/html sincronizados entre os nós.
- `rsync -avz --delete /caminho/origem/ /caminho/destino/:`
 - -a: Modo arquivo (preserva permissões, donos, datas, etc.).
 - -v: Verbose (mostra o progresso).
 - -z: Compressão (reduz o tráfego de rede).
 - --delete: Remove arquivos no destino que não existem mais na origem.

Gerenciamento de Configuração Centralizado (Ansible, Chef, Puppet, SaltStack): ferramentas mais avançadas que permitem declarar o "estado desejado" de seus sistemas em código. Elas automatizam a distribuição e aplicação de configurações em grande escala, garantindo que todos os servidores estejam sempre em conformidade. Ideais para infraestruturas complexas com dezenas ou centenas de servidores.

Orquestradores de Contêineres (KubernetesConfigMaps, Secrets, Persistent Volumes): em ambientes containerizados, orquestradores como Kubernetes oferecem mecanismos nativos para gerenciar configurações (ConfigMaps), dados sensíveis (Secrets) e volumes de dados persistentes. Isso

garante que a configuração correta seja injetada nos contêineres independentemente do nó onde eles são agendados, simplificando a HA em nível de aplicação.

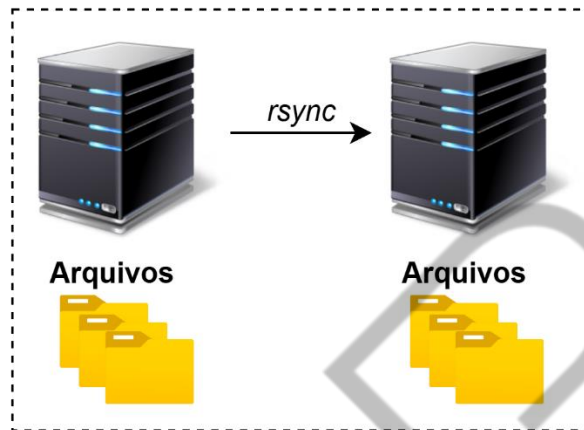


Figura 3 - Fluxo rsync
Fonte: elaborado pelo autor (2025)

Failover Inteligente com Health Checks Avançados: um IP virtual, por si só, não garante que o serviço por trás dele esteja realmente saudável. Nosso Keepalived, por exemplo, pode detectar se o Nginx está *rodando e respondendo HTTP*. Mas e se:

- O Nginx estivesse respondendo 200 OK, mas com erros internos (5xx) para a aplicação real?
- O Nginx estivesse OK, mas o backend que ele serve (ex: um servidor de aplicação, um banco de dados) estivesse fora?
- O disco do servidor estivesse cheio, impactando a performance?

Para um failover verdadeiramente inteligente, precisamos de **Health Checks** mais sofisticados, que vão além de um simples "ping" ou checagem de porta:

Health Checks Ativos: o monitor (e.g., Keepalived, LoadBalancer) *aciona* o serviço em intervalos regulares para verificar seu estado.

Exemplos:

- **HTTP/HTTPS:** fazer uma requisição GET para um endpoint de saúde específico (/healthz ou /status) e esperar um status 200 OK e/ou uma resposta específica que indique a saúde da aplicação;

- **TCP:** tentar abrir uma conexão TCP em uma porta específica;
- **Scripts Personalizados:** executar um script que verifica múltiplos aspectos do sistema (uso de CPU, memória, disco, processos críticos, conexão com banco de dados externo, resposta de APIs internas). Se o script retornar 0 (sucesso), o serviço é considerado saudável. Se retornar 1 (falha), o Keepalived pode acionar o failover.

Health Checks Passivos: o monitor não 'chama' o serviço proativamente, mas *reage a eventos ou mensagens* emitidas por ele ou por outros componentes do sistema.

Exemplos:

- **Monitoramento de Logs:** detectar mensagens de erro críticas ou padrões de falha em logs;
- **Heartbeats:** o serviço envia "sinais de vida" periódicos para um monitor; a ausência desses sinais indica falha;
- **Métricas Anômalas:** detecção de picos de erro, alta latência, ou quedas abruptas de throughput por um sistema de monitoramento centralizado.

A combinação de healthchecks ativos e passivos, configurados em ferramentas como o Keepalived (vrrp_script), permite que o sistema de HA tenha uma visão 360 graus da saúde do serviço, garantindo que o failover ocorra apenas quando há uma falha **real** e impactante, e não em situações transitórias ou irrelevantes.

Testes de Resiliência e ChaosEngineering: Quebrando para Fortalecer

Não basta construir sistemas resilientes; é preciso provar que são. A única forma de ter confiança de que sua arquitetura de HA realmente funciona é **testá-la sob estresse**, simulando falhas de forma controlada. É aqui que entra o conceito de **Testes de Resiliência** e a disciplina do **Chaos Engineering**.

Testes de Resiliência: prática de injetar falhas em um sistema de forma controlada para identificar pontos fracos e verificar como o sistema reage. O objetivo é validar que os mecanismos de failover, recuperação e redundância funcionam conforme o esperado.

ChaosEngineering: é uma disciplina mais ampla que vai além do teste. Seu objetivo é aprender sobre o comportamento de um sistema em ambientes turbulentos, buscando proativamente fraquezas para mitigá-las antes que causem interrupções em produção.

Princípios:

1. **Formular uma Hipótese:**ex: "Se o servidor de banco de dados cair, a aplicação continuará a funcionar com dados em cache por 5 minutos.";
2. **Planejar um Experimento:** definir a falha a ser injetada e como medir o impacto;
3. **Executar o Experimento:** injetar a falha em um ambiente controlado (pode ser produção, mas com precaução);
4. **Verificar e Aprender:** analisar os resultados, comparar com a hipótese e documentar as lições aprendidas.

Exemplos de Falhas a Simular:

- Parar ou reiniciar serviços (Nginx, Keepalived, bancos de dados, aplicações);
- Desconectar interfaces de rede ou introduzir latência;
- Consumir recursos (CPU, memória, disco) para simular sobrecarga;
- Degradar a performance de dependências externas.

Se você não testar a resiliência do seu sistema de forma controlada, a falha vai testá-lo em produção, e geralmente no pior momento possível. A simulação proativa permite que você descubra e corrija problemas em um ambiente seguro, fortalecendo sua arquitetura e a confiança da equipe.

Segurança na Camada de Aplicação: Blindando a Borda

Com a Alta Disponibilidade e a Resiliência em mente, é impossível ignorar a **Segurança**. De que adianta um sistema que está sempre no ar se ele estiver vulnerável a ataques? A segurança cibernética evolui constantemente, e as aplicações web são um alvo prioritário.

Web Application Firewall (WAF): O Guarda-Costas da Camada 7

Um **Web Application Firewall (WAF)** é um tipo específico de firewall que monitora, filtra e bloqueia o tráfego HTTP/HTTPS entre um aplicativo web e a internet. Sua principal característica é operar na **camada de aplicação (Camada 7)** do modelo OSI, diferente dos firewalls de rede tradicionais que atuam nas Camadas 3 e 4 (rede e transporte).

Por que um WAF é diferente de um Firewall Tradicional?

- Um firewall de rede tradicional (Camadas 3/4) é excelente para proteger contra ataques em nível de rede (scans de portas, inundações SYN, DDoS volumétrico). No entanto, ele não consegue "entender" o conteúdo de uma requisição HTTP. Ele não sabe se um campo de login contém um SQL Injection ou um script malicioso;
- O WAF, operando na Camada 7, **entende a lógica da aplicação**. Ele analisa o conteúdo, os parâmetros, os cabeçalhos HTTP, os cookies e o corpo das requisições e respostas. Isso permite que ele proteja contra ataques que visam vulnerabilidades no código ou na lógica de negócio da aplicação.

Principais Funções de um WAF:

- **Proteção contra OWASP Top 10:** SQL Injection, Cross-Site Scripting (XSS), Broken Authentication, Sensitive Data Exposure, etc;
- **Mitigação de Bots:** bloqueio de web scraping, credential stuffing, ataques de força bruta automatizados;
- **Validação de Entrada:** garante que os dados enviados para a aplicação estejam no formato esperado e não contenham cargas maliciosas.

Tipos de WAF:

- **WAF Dedicado (Hardware/Software Appliance):** dispositivos físicos ou máquinas virtuais dedicadas, oferecendo alta performance e recursos avançados. Exemplos: F5 BIG-IP ASM, Barracuda WAF;
- **WAF Baseado em Nuvem (Cloud WAF):** serviços gerenciados por provedores de nuvem ou CDNs, oferecendo escalabilidade, simplicidade e

proteção distribuída contra DDoS. Exemplos: AWS WAF, Cloudflare WAF, Akamai;

- **WAF Integrado (Módulo de Servidor Web):** módulos instalados diretamente no servidor web (Apache, Nginx, IIS). Geralmente são open source e oferecem alto controle e fácil integração, embora consumam recursos do servidor. Exemplo: **ModSecurity (nosso foco!)**.

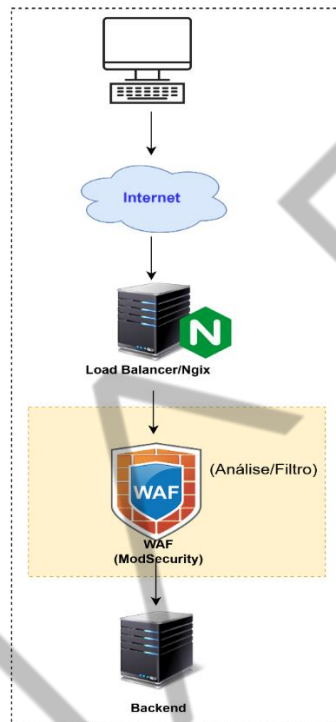


Figura 4- WAF
Fonte: elaborado pelo autor (2025)

ModSecurity e OWASP Core Rule Set (CRS): A Dupla Dinâmica

Para implementar um WAF de forma granular e com alto controle, a combinação de ModSecurity e OWASP CRS é uma escolha popular e poderosa, especialmente com Nginx.

ModSecurity:

- É um **motor WAF de código aberto** que opera como um módulo em servidores web (Nginx, Apache, IIS) ou como um proxy reverso;
- Ele analisa requisições e respostas HTTP usando um conjunto de regras (ruleset) e pode bloquear, alertar, registrar ou até mesmo redirecionar tráfego com base nessas regras;

- Sua flexibilidade vem de sua linguagem de regras própria, que permite definir padrões de ataques, expressões regulares e condições complexas.

OWASP Core Rule Set (CRS):

- É um **conjunto de regras genéricas e de propósito geral** para o ModSecurity, mantido pela OWASP (Open Web Application Security Project), uma organização sem fins lucrativos focada em segurança de aplicações web;
- O CRS é projetado para proteger aplicações web contra uma vasta gama de ataques, incluindo todos os listados no **OWASP Top 10** (as 10 vulnerabilidades de segurança mais críticas em aplicações web, como SQL Injection, XSS, etc.);
- A filosofia do CRS é detectar padrões comuns de ataques, reduzindo a necessidade de regras específicas para cada aplicação. Ao integrá-lo ao ModSecurity, você tem uma proteção inicial robusta "pronta para usar".

Como o WAF (ModSecurity + CRS) Protege?

- **Interceptação:** toda requisição HTTP/HTTPS destinada ao Nginx é interceptada pelo módulo ModSecurity;
- **Análise de Regras:** o ModSecurity compara a requisição com as regras do OWASP CRS e quaisquer regras personalizadas. Ele procura por:
 - **PatternMatching:** strings ou padrões conhecidos de ataques (ex: ' OR 1=1-- para SQL Injection);
 - **AnomalyDetection:** comportamentos incomuns ou inesperados na requisição que podem indicar uma tentativa de ataque.
- **Pontuação de Ameaça (Anomaly Scoring):** cada regra acionada aumenta uma "pontuação de anomalia". Se essa pontuação exceder um limite pré-definido, a requisição é considerada maliciosa.

Ação:

- **BLOQUEIA:** se a pontuação for alta, o WAF bloqueia a requisição (geralmente com um erro 403 Forbidden);

- **REGISTRA:** todas as requisições (especialmente as bloqueadas) são registradas em logs para auditoria e análise de segurança;
- **ALERTA:** pode emitir alertas para sistemas de monitoramento de segurança (SIEM).

Boas Práticas de Segurança no Nginx: Um Escudo Multicamadas

Além de um WAF, a segurança de uma arquitetura web depende de uma série de boas práticas, muitas das quais podem ser implementadas diretamente no Nginx.

TLS/SSL (Transport Layer Security / Secure Sockets Layer): Criptografia Essencial

- **Importância:** criptografa a comunicação entre o cliente e o servidor, protegendo dados sensíveis de interceptação (ataques "Man-in-the-Middle"). O "cadeadinho verde" no navegador é a representação visual disso;
- **Sempre HTTPS:** todas as aplicações web, sem exceção, devem usar HTTPS;
- **Versões Seguras:** priorize as versões mais recentes do TLS (1.2 e 1.3). Desabilite versões antigas (TLS 1.0, 1.1, SSLv2, SSLv3) que possuem vulnerabilidades conhecidas;
- **Ciphers Fortes:** utilize apenas suítes de criptografia (ciphers) modernas e robustas;
- **HSTS (HTTP Strict Transport Security):** um cabeçalho de segurança que força o navegador a sempre se comunicar via HTTPS para aquele domínio, mesmo que o usuário tente acessar via HTTP. Isso previne ataques de downgrade e garante que a comunicação seja sempre criptografada.

Rate Limiting: Controlando o Fluxo de Requisições

- **Definição:** técnica para limitar o número de requisições que um cliente (geralmente por IP ou por sessão) pode fazer a um servidor em um determinado período.

- **Objetivo:** Proteger contra:
 - **DDoS (Distributed Denial of Service):** mitiga a sobrecarga por inundações de requisições;
 - **Força Bruta:** impede tentativas massivas de login ou adivinhação de senhas;
 - **Web Scraping:** dificulta a extração automatizada e abusiva de conteúdo.
- **Nginx:** possui diretivas nativas (limit_req_zone, limit_req) para configurar limites de taxa de forma muito eficiente.

Monitoramento e Alertas: Os Olhos e Ouvidos da Segurança

- **Importância:** detectar proativamente anomalias, ataques, falhas e problemas de performance. A visibilidade é a chave para a segurança.
- **O que Monitorar:**
 - **Logs:** acesso (access.log), erros (error.log), e, crucialmente, logs do ModSecurity (registrando os bloqueios e regras acionadas);
 - **Métricas de Performance:** requisições por segundo, latência, erros (códigos 5xx), uso de CPU/Memória;
 - **Métricas de Segurança:** número de bloqueios do WAF, tentativas de login falhas, tráfego vindo de IPs suspeitos.
- **Ferramentas:** Prometheus + Grafana (para métricas e dashboards), ELK Stack (Elasticsearch, Logstash, Kibana) para agregação e análise de logs, Datadog/New Relic para observabilidade completa;
- **Alertas:** configurar notificações automatizadas (Slack, e-mail, PagerDuty) para eventos críticos de segurança, garantindo uma resposta rápida a incidentes.

Testes de Recuperação de Desastres (DR - Disaster Recovery): Preparando-se para o Pior Cenário

HA vs. DR:

- **HA (Alta Disponibilidade):** foca em resiliência a falhas locais (servidor, rack, zona de disponibilidade) para *manter-se no ar*;
- **DR (Recuperação de Desastres):** foca em resiliência a falhas de larga escala (datacenter inteiro, região geográfica, ataque cibernético massivo) para *recuperar-se do desastre*.
 - **Importância:** validar a capacidade de restaurar os serviços após um evento catastrófico. "Não é sobre 'se' um desastre vai acontecer, mas 'quando'.;
 - **Componentes de um Plano de DR:** backup e restauração de dados, estratégias de replicação geográfica (multirregião), failover para outra região/site;
 - **Testes Regulares:** crucialmente, testar o plano de DR regularmente para medir o RTO (Recovery Time Objective – tempo máximo de inatividade aceitável) e RPO (Recovery Point Objective – perda máxima de dados aceitável). Seu ambiente DR deve ser tão seguro e robusto quanto o primário.

MERCADO, CASES E TENDÊNCIAS

Site Reliability Engineering: How Google Runs Production Systems – autores Betsy Beyer, Chris Jones, Jennifer Petoff, Niall Murphy (O'Reilly, 2016)

Considerado o texto seminal da Engenharia de Confiabilidade de Site (SRE), apresenta os princípios que permitem ao Google escalar e manter sistemas de alta disponibilidade. Introduz conceitos como automação, error budgets e resposta a incidentes, trazendo insights aplicáveis também para organizações menores.

The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations – autores Gene Kim, Jez Humble, Patrick Debois, John Willis (IT Revolution Press, 2016)

É um guia prático para aplicar cultura e práticas DevOps. Mostra como integrar agilidade, confiabilidade e segurança em todo o ciclo de vida de sistemas, tornando-se leitura essencial para quem busca alinhar desenvolvimento e operação em ambientes modernos.

The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws – autores Dafydd Stuttard, Marcus Pinto (editora Wiley, 2011)

Clássico em segurança de aplicações web, ensina a mentalidade do atacante e técnicas como SQL Injection e XSS. Mesmo não focado em WAFs, fornece base indispensável para configurar, testar e validar defesas como ModSecurity e OWASP CRS.

O QUE VOCÊ VIU NESTA AULA?

Nesta jornada, você desvendou os fundamentos da Alta Disponibilidade (HA), compreendendo o custo real do downtime e a importância de métricas como SLA e SLO para o negócio. Exploramos as estratégias Ativo-Ativo e Ativo-Passivo, aprendendo a identificar e mitigar Pontos Únicos de Falha (SPOFs), com um foco prático na configuração de clusters Nginx com Keepalived. Aprofundamos a resiliência com a replicação de configurações e healthchecks inteligentes, capazes de acionar failovers automáticos e validar a robustez de sistemas por meio de testes de resiliência.

Por fim, mergulhamos na segurança de aplicações, implementando Web Application Firewalls (WAFs) como ModSecurity e OWASP CRS no Nginx, e aplicando boas práticas essenciais como TLS, Rate Limiting e monitoramento, consolidando seu conhecimento para construir arquiteturas web não apenas disponíveis, mas também seguras e preparadas para os desafios do mundo real.

REFERÊNCIAS

DOCKER. **Docker Compose Documentation.** 2025. Disponível em: <https://docs.docker.com/compose/>. Acesso em: 04 set. 2025.

DOCKER. **Docker Documentation.** 2025. Disponível em: <https://docs.docker.com/>. Acesso em: 04 set. 2025.

GRAFANA LABS. **Grafana Documentation.** 2025. Disponível em: <https://grafana.com/docs/>. Acesso em: 04 set. 2025.

KEEPALIVED. **Keepalived Overview.** 2025. Disponível em: <https://www.keepalived.org/>. Acesso em: 04 set. 2025.

MODSECURITY. **ModSecurity Documentation.** 2025. Disponível em: <https://github.com/SpiderLabs/ModSecurity>. Acesso em: 04 set. 2025.

NETFLIX TECHNOLOGY BLOG. **Chaos Engineering Upgraded.** 2019. Disponível em: <https://techblog.netflix.com/2019/01/chaos-engineering-upgraded.html>. Acesso em: 04 set. 2025.

NGINX. **Nginx Official Documentation.** 2025. Disponível em: <https://nginx.org/en/docs/>. Acesso em: 04 set. 2025.

OWASP. **OWASP Core Rule Set (CRS) Project.** 2025. Disponível em: <https://owasp.org/www-project-modsecurity-core-rule-set/>. Acesso em: 04 set. 2025.

OWASP. **OWASP Top 10 - 2021.** 2025. Disponível em: <https://owasp.org/Top10/>. Acesso em: 04 set. 2025.

PROMETHEUS. **Prometheus Documentation.** 2025. Disponível em: <https://prometheus.io/docs/>. Acesso em: 04 set. 2025.

RSYNC. **Rsync Man Page.** 2025. Disponível em: <https://linux.die.net/man/1/rsync>. Acesso em: 04 set. 2025.

TRANSPORT LAYER SECURITY (TLS). **IETF (Internet Engineering Task Force) RFCs. TLS 1.3.** Disponível em: <https://www.rfc-editor.org/rfc/rfc8446>. Acesso em: 04 set. 2025.

PALAVRAS-CHAVE

Alta Disponibilidade. Segurança de Aplicações. DevOps.

EMSE



POSTECH